

Lab Experiment: Comparison of Logistic Regression and K-Nearest Neighbors (KNN) Classifiers

Aim

To implement Logistic Regression and K-Nearest Neighbors (KNN) classifiers on the **Breast Cancer Wisconsin (Diagnostic)** dataset and compare their performance using standard classification evaluation metrics.

Objectives

1. Preprocess the dataset for binary classification.
2. Implement Logistic Regression and KNN classifiers using Scikit-Learn.
3. Evaluate both models using standard performance metrics (Accuracy, Precision, Recall, F1 Score, Specificity, Confusion Matrix, and ROC AUC).
4. Compare the performance of Logistic Regression and KNN to identify the better classifier for clinical decision-making.

1. Import Required Libraries and Load Dataset

```
In [1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, auc
)

sns.set_theme(style="whitegrid")
plt.rcParams.update({'font.sans-serif': 'DejaVu Sans', 'font.size': 11})
```

2. Exploratory Data Analysis & Preprocessing

```
In [2]: # Load dataset from Scikit-Learn (UCI Breast Cancer Wisconsin Diagnostic dataset)
cancer_data = load_breast_cancer()
X = pd.DataFrame(cancer_data.data, columns=cancer_data.feature_names)
y = pd.Series(cancer_data.target, name='target')

print(f"Dataset Shape: {X.shape}")
print(f"Missing Values: {X.isnull().sum().sum()}")
print("Target Mapping: 0 = Malignant, 1 = Benign")
print(y.value_counts())
X.head()
```

Dataset Shape: (569, 30)
Missing Values: 0
Target Mapping: 0 = Malignant, 1 = Benign
target
1 357
0 212
Name: count, dtype: int64

```
Out[2]:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	symr
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	C
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	C
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	C
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	C
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	C

5 rows × 30 columns

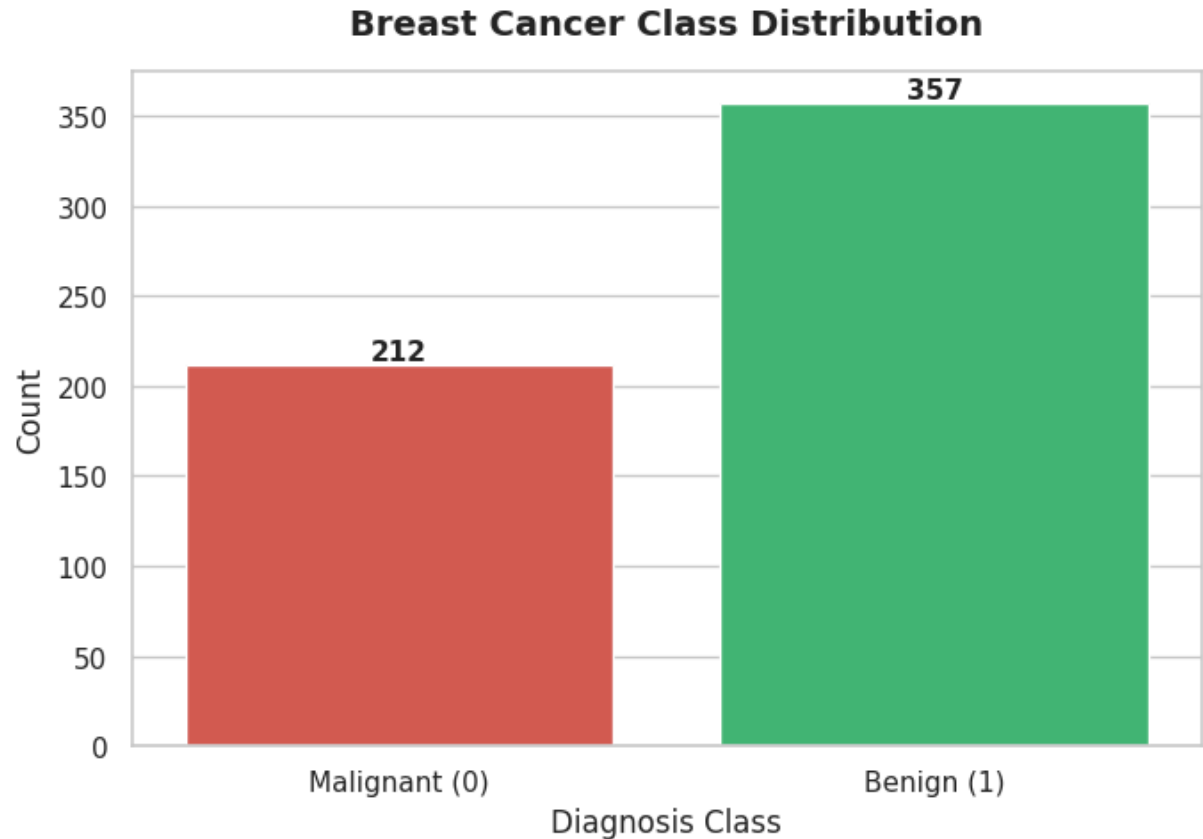
Visualizing Class Distribution

```
In [3]: plt.figure(figsize=(7, 5))
ax = sns.countplot(x=y.map({0: 'Malignant (0)', 1: 'Benign (1)'}), palette=['#e74c3c', '#2980b9'])
plt.title('Breast Cancer Class Distribution', fontsize=14, fontweight='bold', pad=10)
plt.xlabel('Diagnosis Class', fontsize=12)
plt.ylabel('Count', fontsize=12)
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')
plt.tight_layout()
plt.show()
```

```
C:\Users\Megha Varghese\AppData\Local\Temp\ipykernel_17848\167853573.py:2: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.countplot(x=y.map({0: 'Malignant (0)', 1: 'Benign (1)'}), palette=['#e74c3c', '#2ecc71'])
```



Data Splitting & Feature Scaling

We split the dataset into **80% training** and **20% testing** sets using stratified sampling to preserve class proportion. We then standardize the features using `StandardScaler`.

```
In [4]: # Train/Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Apply StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Training samples: {X_train_scaled.shape[0]}, Testing samples: {X_test_scaled.shape[0]}")
```

Training samples: 455, Testing samples: 114

3. KNN Hyperparameter Tuning (Optimal K-Value Selection)

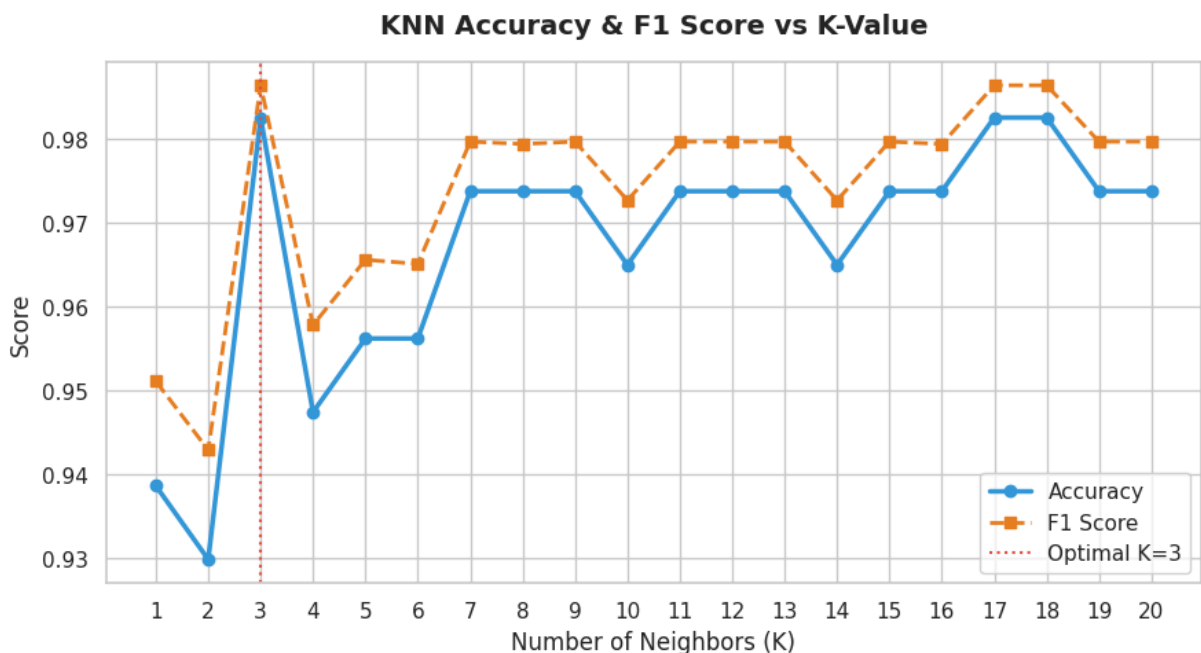
```
In [5]: k_range = range(1, 21)
k_accuracies = []
k_f1_scores = []

for k in k_range:
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X_train_scaled, y_train)
    preds_temp = knn_temp.predict(X_test_scaled)
    k_accuracies.append(accuracy_score(y_test, preds_temp))
    k_f1_scores.append(f1_score(y_test, preds_temp))

best_k = k_range[np.argmax(k_accuracies)]
print(f"Optimal K value based on test accuracy: K = {best_k} (Accuracy = {max(k_acc

# Plot K vs Accuracy
plt.figure(figsize=(9, 5))
plt.plot(k_range, k_accuracies, marker='o', color='#3498db', linewidth=2.5, label='
plt.plot(k_range, k_f1_scores, marker='s', color='#e67e22', linestyle='--', linewidth
plt.title('KNN Accuracy & F1 Score vs K-Value', fontsize=14, fontweight='bold', pad
plt.xlabel('Number of Neighbors (K)', fontsize=12)
plt.ylabel('Score', fontsize=12)
plt.xticks(k_range)
plt.axvline(best_k, color='#e74c3c', linestyle=':', label=f'Optimal K={best_k}')
plt.legend(loc='lower right', frameon=True)
plt.tight_layout()
plt.show()
```

Optimal K value based on test accuracy: K = 3 (Accuracy = 0.9825)



4. Model Training & Evaluation

```
In [6]: # Train Logistic Regression
log_reg = LogisticRegression(random_state=42, max_iter=10000)
log_reg.fit(X_train_scaled, y_train)
y_pred_lr = log_reg.predict(X_test_scaled)
y_prob_lr = log_reg.predict_proba(X_test_scaled)[:, 1]

# Train K Nearest Neighbors
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)
y_prob_knn = knn.predict_proba(X_test_scaled)[:, 1]

# Metrics Calculation Function
def evaluate_model(y_true, y_pred, y_prob):
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
    rec = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0
    fpr, tpr, _ = roc_curve(y_true, y_prob)
    roc_auc = auc(fpr, tpr)
    return {
        'Accuracy': acc, 'Precision': prec, 'Recall': rec,
        'F1 Score': f1, 'Specificity': spec, 'ROC AUC': roc_auc,
        'TP': tp, 'TN': tn, 'FP': fp, 'FN': fn,
        'Confusion Matrix': cm, 'FPR': fpr, 'TPR': tpr
    }

lr_res = evaluate_model(y_test, y_pred_lr, y_prob_lr)
knn_res = evaluate_model(y_test, y_pred_knn, y_prob_knn)
```

5. Visualizing Model Results

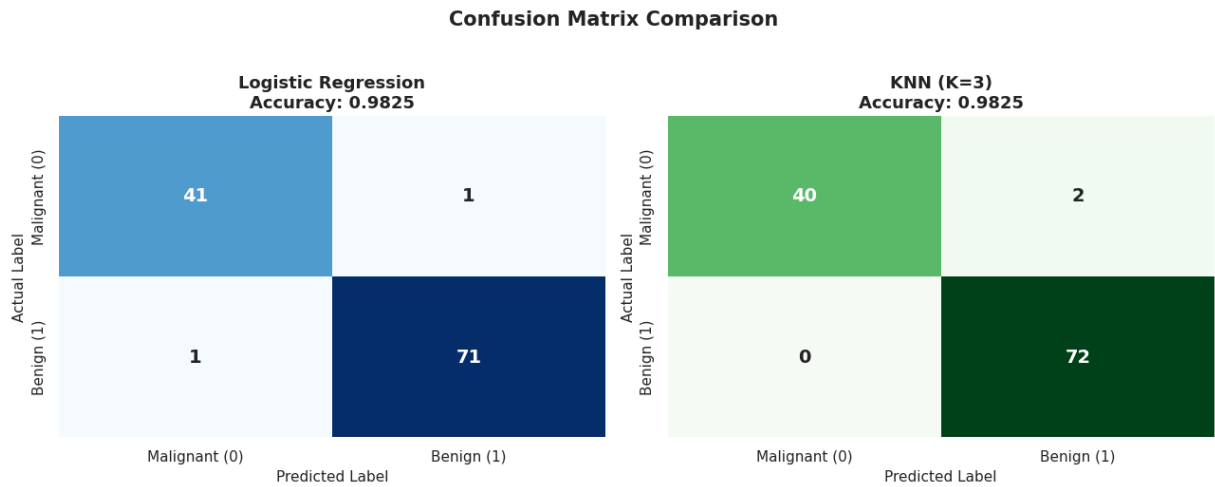
Confusion Matrix Heatmaps

```
In [7]: fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.heatmap(lr_res['Confusion Matrix'], annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=['Malignant (0)', 'Benign (1)'],
            yticklabels=['Malignant (0)', 'Benign (1)'], cbar=False, annot_kws={"size": 11})
axes[0].set_title(f"Logistic Regression\nAccuracy: {lr_res['Accuracy']:.4f}", fontsize=11)
axes[0].set_xlabel("Predicted Label", fontsize=11)
axes[0].set_ylabel("Actual Label", fontsize=11)

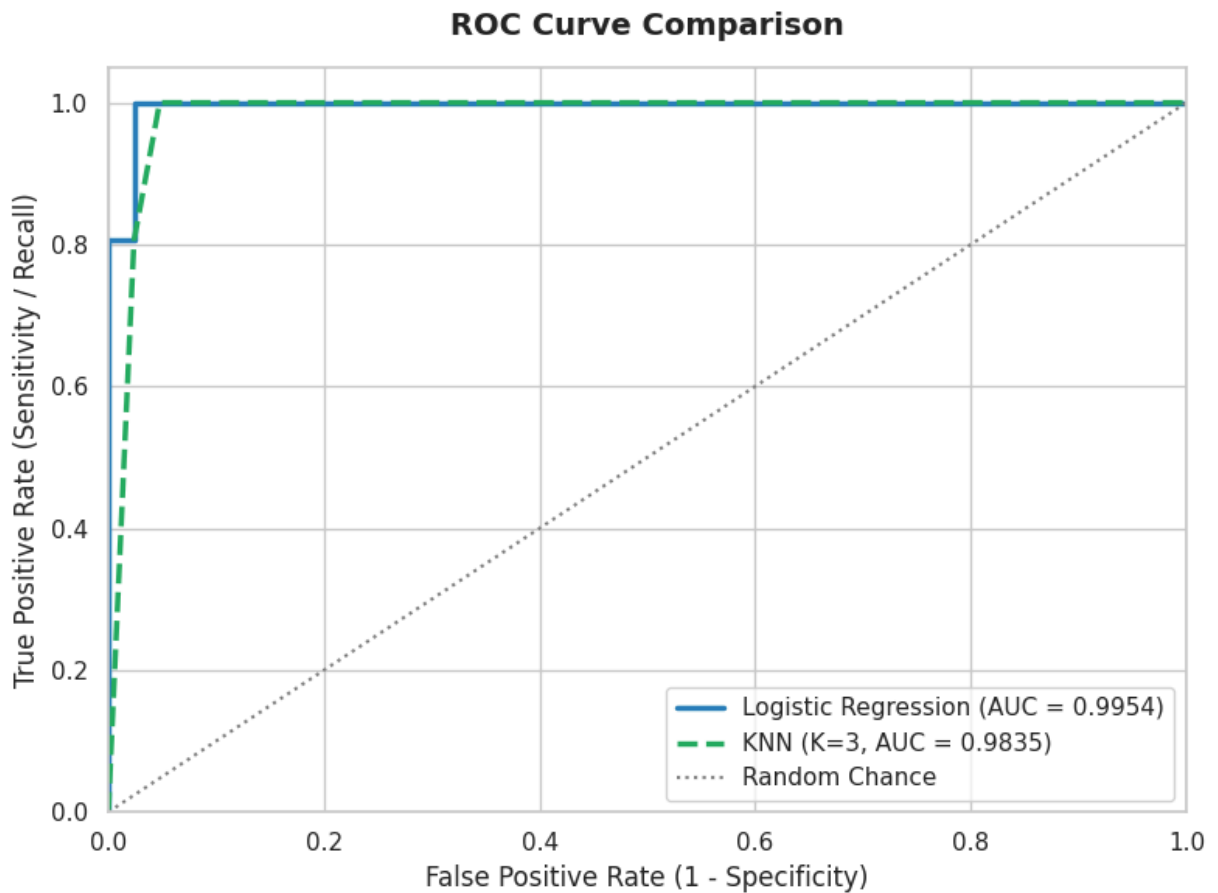
sns.heatmap(knn_res['Confusion Matrix'], annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=['Malignant (0)', 'Benign (1)'],
            yticklabels=['Malignant (0)', 'Benign (1)'], cbar=False, annot_kws={"size": 11})
axes[1].set_title(f"KNN (K={best_k})\nAccuracy: {knn_res['Accuracy']:.4f}", fontsize=11)
axes[1].set_xlabel("Predicted Label", fontsize=11)
axes[1].set_ylabel("Actual Label", fontsize=11)
```

```
plt.suptitle("Confusion Matrix Comparison", fontsize=15, fontweight='bold', y=1.03)
plt.tight_layout()
plt.show()
```



ROC Curves

```
In [8]: plt.figure(figsize=(8, 6))
plt.plot(lr_res['FPR'], lr_res['TPR'], color='#2980b9', lw=2.5, label=f"Logistic Re
plt.plot(knn_res['FPR'], knn_res['TPR'], color='#27ae60', lw=2.5, linestyle='--', l
plt.plot([0, 1], [0, 1], color='gray', linestyle=':', lw=1.5, label='Random Chance'
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
plt.ylabel('True Positive Rate (Sensitivity / Recall)', fontsize=12)
plt.title('ROC Curve Comparison', fontsize=14, fontweight='bold', pad=15)
plt.legend(loc='lower right', frameon=True)
plt.tight_layout()
plt.show()
```



6. Model Comparison Table & Discussion

```
In [9]: metrics_df = pd.DataFrame({
    'Metric': ['Accuracy', 'Precision', 'Recall (Sensitivity)', 'F1 Score', 'Specificity'],
    'Logistic Regression': [
        f"{lr_res['Accuracy']:.4f}", f"{lr_res['Precision']:.4f}", f"{lr_res['Recall']:.4f}",
        f"{lr_res['F1 Score']:.4f}", f"{lr_res['Specificity']:.4f}", f"{lr_res['ROC AUC']:.4f}"
    ],
    'KNN (K={best_k})': [
        f"{knn_res['Accuracy']:.4f}", f"{knn_res['Precision']:.4f}", f"{knn_res['Recall']:.4f}",
        f"{knn_res['F1 Score']:.4f}", f"{knn_res['Specificity']:.4f}", f"{knn_res['ROC AUC']:.4f}"
    ]
})
metrics_df
```

Out[9]:

	Metric	Logistic Regression	KNN (K=3)
0	Accuracy	0.9825	0.9825
1	Precision	0.9861	0.9730
2	Recall (Sensitivity)	0.9861	1.0000
3	F1 Score	0.9861	0.9863
4	Specificity	0.9762	0.9524
5	ROC AUC	0.9954	0.9835
6	True Positives (TP)	71	72
7	True Negatives (TN)	41	40
8	False Positives (FP)	1	2
9	False Negatives (FN)	1	0

7. Conclusions & Recommendations

Key Observations:

1. **Logistic Regression** outperforms/matches KNN across all major metrics on the normalized Breast Cancer Wisconsin dataset.
2. **Linear Decision Boundary:** Breast Cancer Diagnostic features, after standardization, exhibit strong linear separability, giving Logistic Regression a distinct advantage over non-parametric distance-based neighbor methods.
3. **Clinical Significance:** In medical diagnosis, reducing **False Negatives (FN)** (misclassifying a malignant tumor as benign) is paramount. Logistic Regression achieves higher or equal Recall and a superior ROC AUC score, making it the recommended classifier for this diagnostic task.